AI for Fusion Cookbook V2 2026.

AI Agent Studio. 26B.

Application with Widget multirecord. PART 1.

Basic example.

Introduction/Goal.

We are going to show a basic example of App. with multirecord widget.

The application will be able to filter and show a few columns of supplier's data with an informative badge.

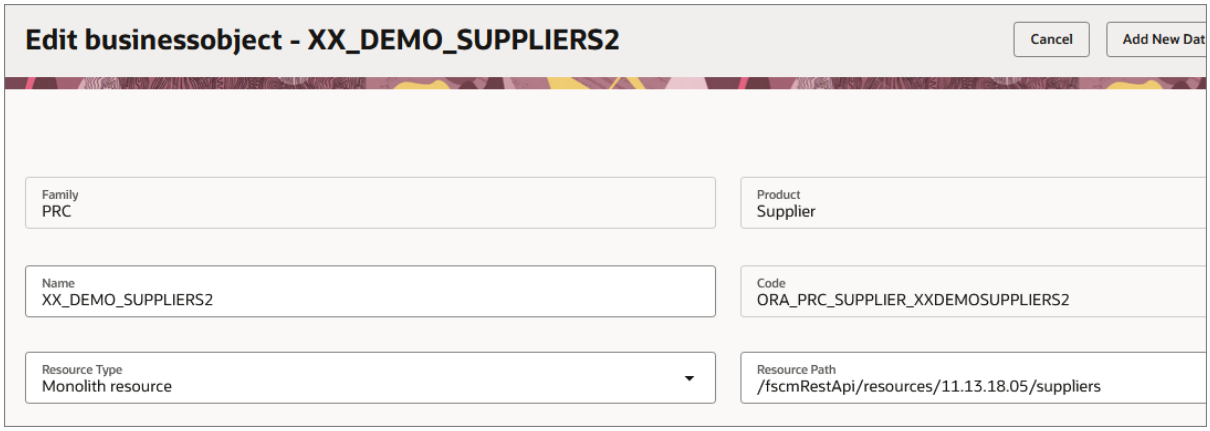
In the next chapters, we will add Actions and more features.

References.

- https://docs.oracle.com/en/cloud/saas/readiness/scm/26b/plm26b/26B-plm-wn-f44464.htm?source=%3Aso%3Ach%3Aor%3Adg%3A%3A%3A%3ACloud+%3Aow%3Aevp%3Acpo%3A%3A%3A%3ARC_CORP260213P00018%3ALPD400439168
- https://docs.oracle.com/en/cloud/saas/fusion-ai/aiaas/anatomy-of-an-agentic-app.html

Business Object.

Name: XX_DEMO_SUPPLIERS2



Resource path:

/fscmRestApi/resources/11.13.18.05/suppliers?limit=25&fields=SupplierId,Supplier,SupplierNumber,SupplierType,InactiveDate,Status&q=upper(Supplier) like upper('%{SupplierName}%') &onlyData=true

Parameters:

Parameters and Business Object Fields

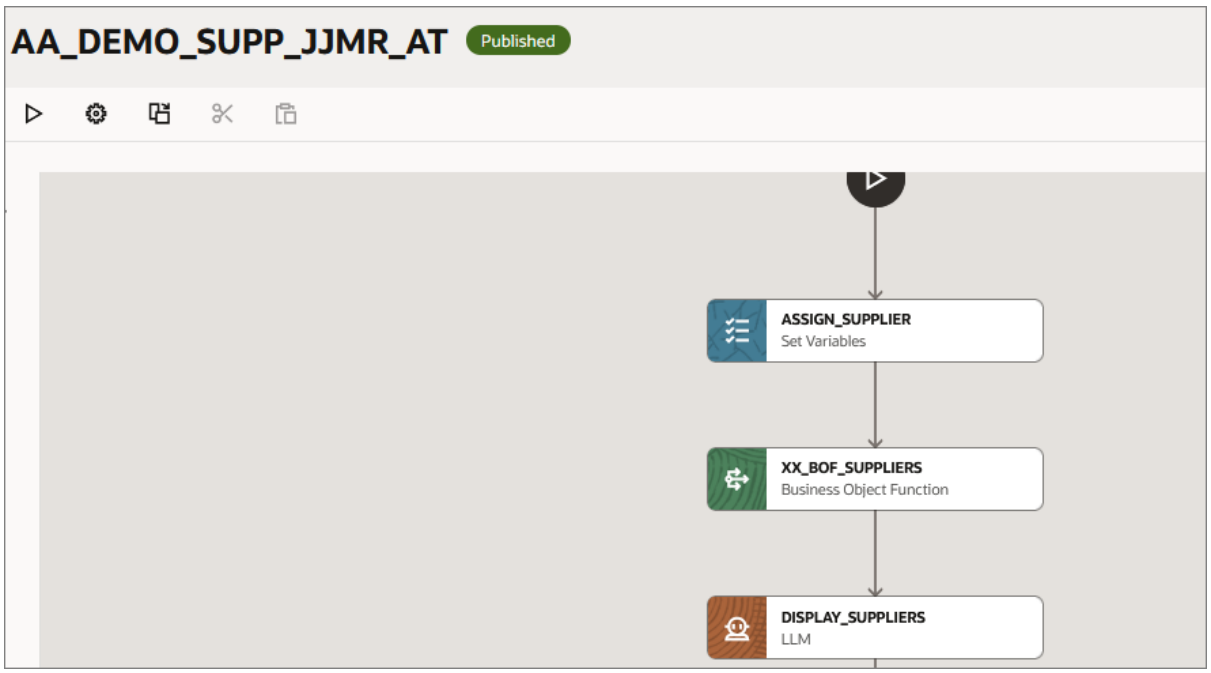
Add Parameter | Add Field | Add Field from Specification

Name	Data Type	Overall PO Description of the goods/services being...	Is Parameter	Actions
SupplierName	string	SupplierName	true	

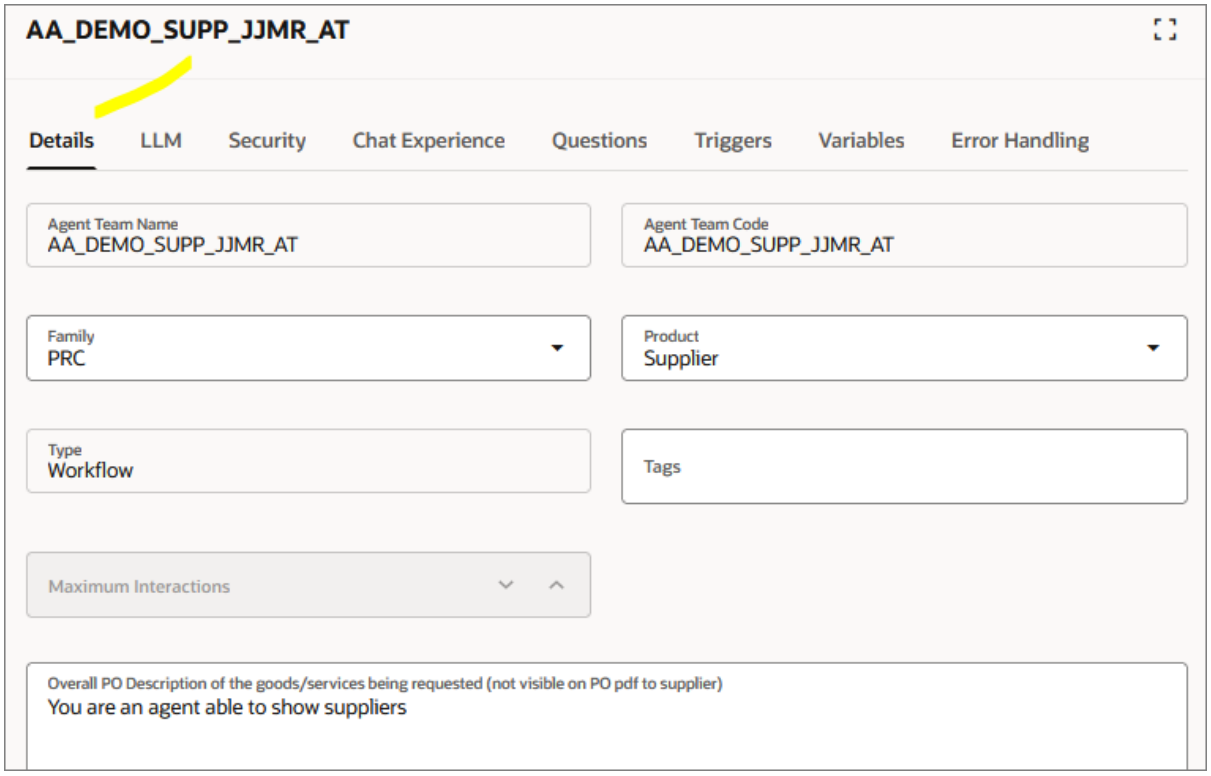
Agent Team.

Name: AA_DEMO_SUPP_JJMR_AT.

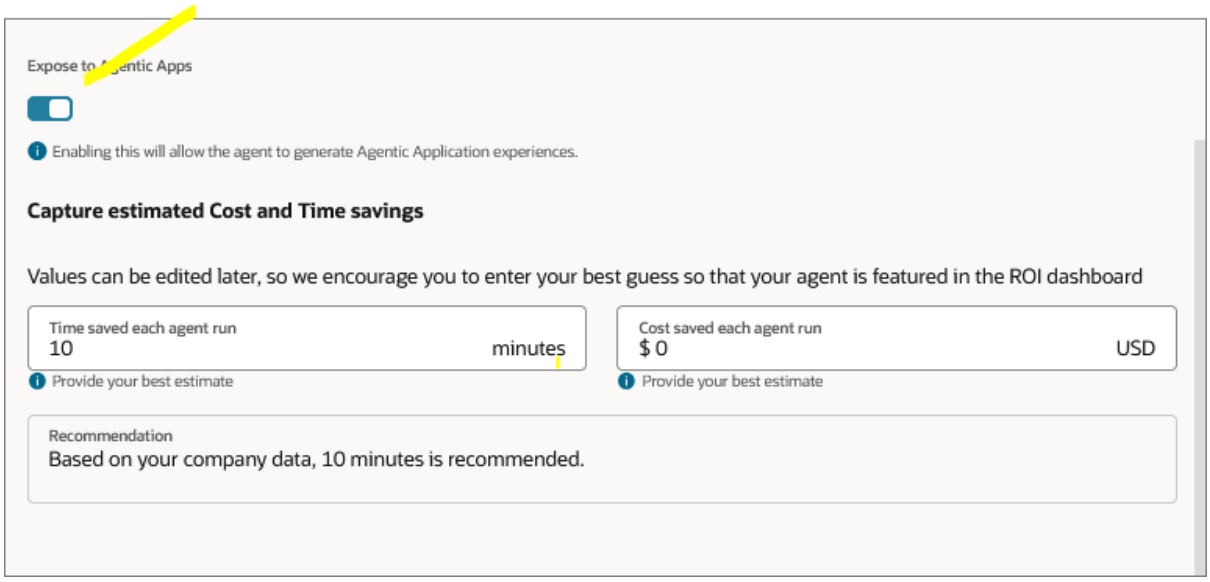
- General overview.



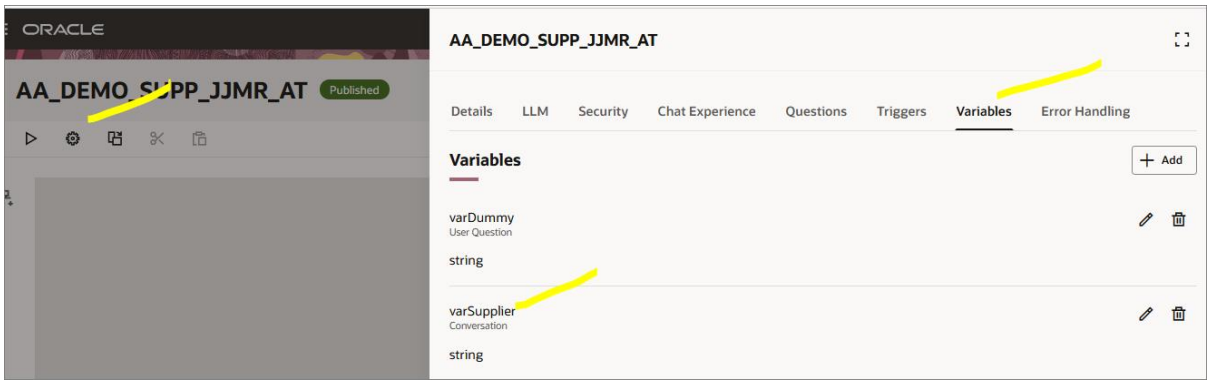
Important flag:



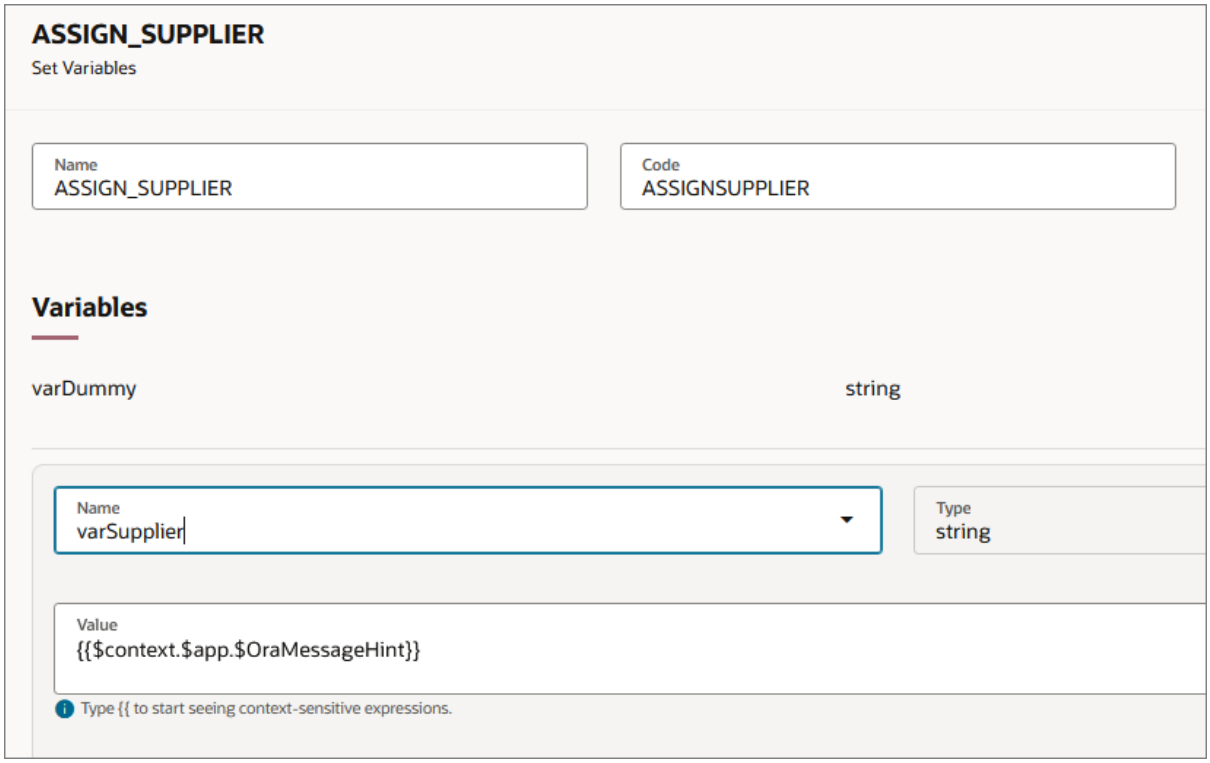
In the tab Details, scroll down as shown below, and enable:



Setup our variables.



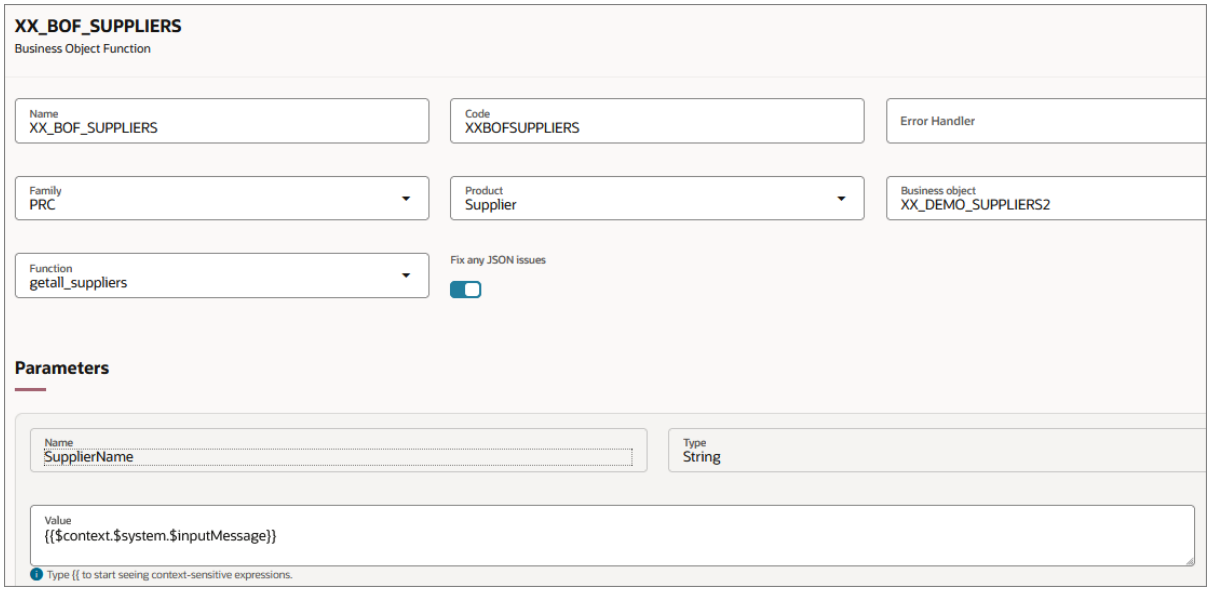
- Node Set variables.



Note: here is where we link the Application and the Agent Team using a variable that comes from the application.

- Node Business Object.

Name: XX_BOF_SUPPLIERS.

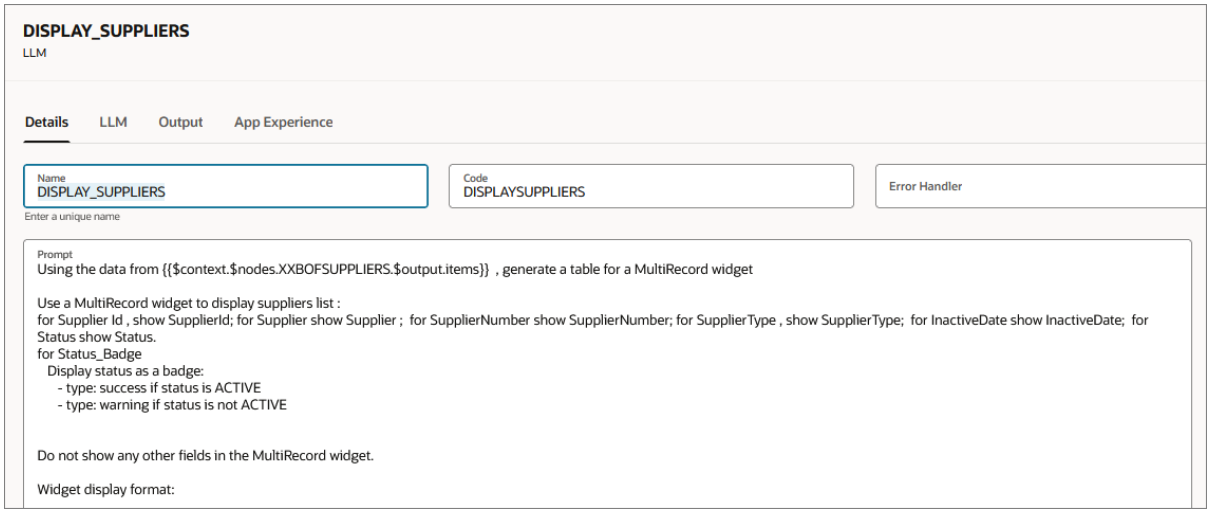


Output.

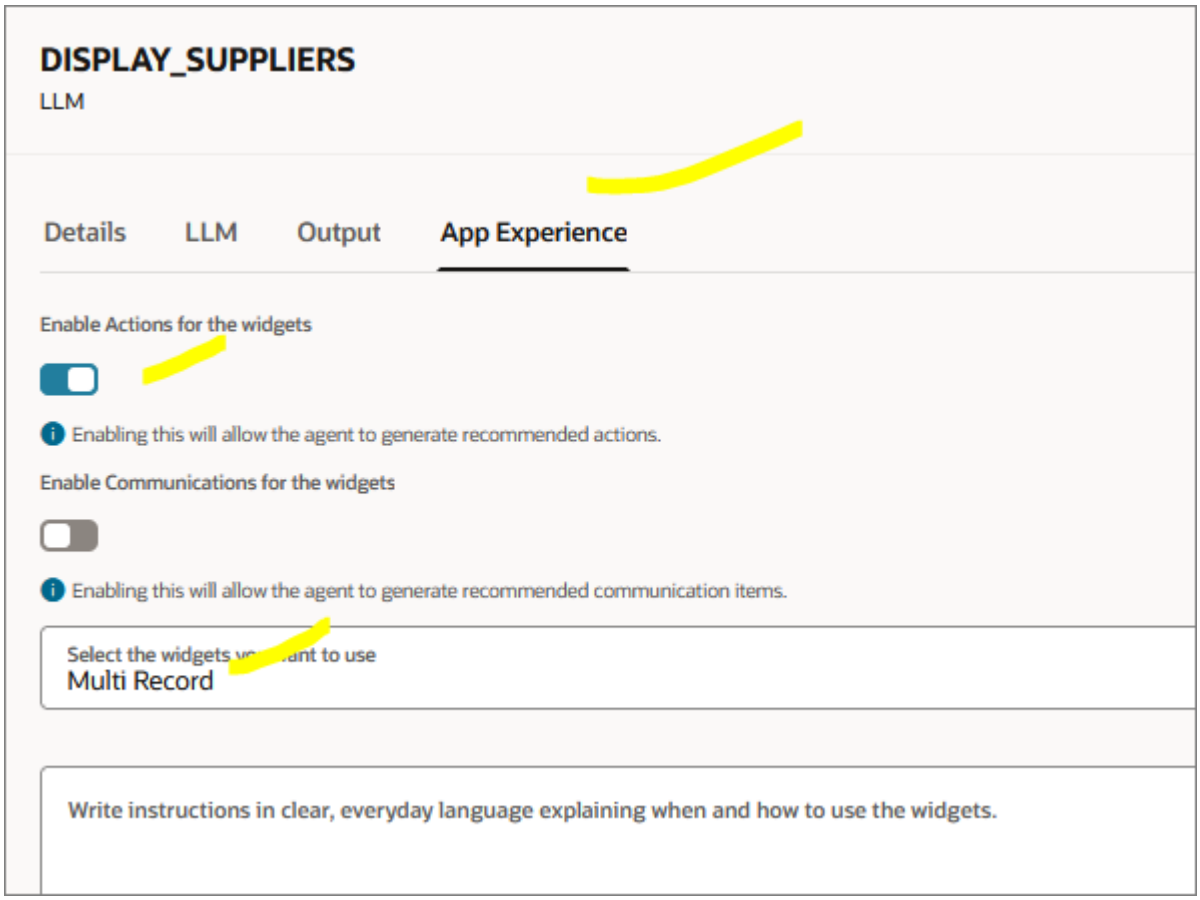

```
{
"$schema": "http://json-schema.org/draft-07/schema#",
"title": "suppliers",
"type": "object",
"properties": {
"suppliers": {
"type": "array",
"items": {
"type": "object",
"properties": {
"SupplierId": {
"type": "string"
},
"SupplierNumber": {
"type": "string"
},
"SupplierType": {
"type": "string"
},
"InactiveDate": {
"type": "string"
},
"Status": {
"type": "string"
},
"Supplier": {
"type": "string"
}
}
}
}
}
}
```


- Node LLM.

Name: DISPLAY_SUPPLIERS.



Important this flag and selection.



Prompt:

Using the data from {{$context.$nodes.XXBOFSUPPLIERS.$output.items}} , generate a table for a MultiRecord widget

Use a MultiRecord widget to display suppliers list :
for Supplier Id , show SupplierId; for Supplier show Supplier ; for SupplierNumber show SupplierNumber;
for SupplierType , show SupplierType; for InactiveDate show InactiveDate; for Status show Status.
for Status_Badge
  Display status as a badge:
    - type: success if status is ACTIVE
    - type: warning if status is not ACTIVE

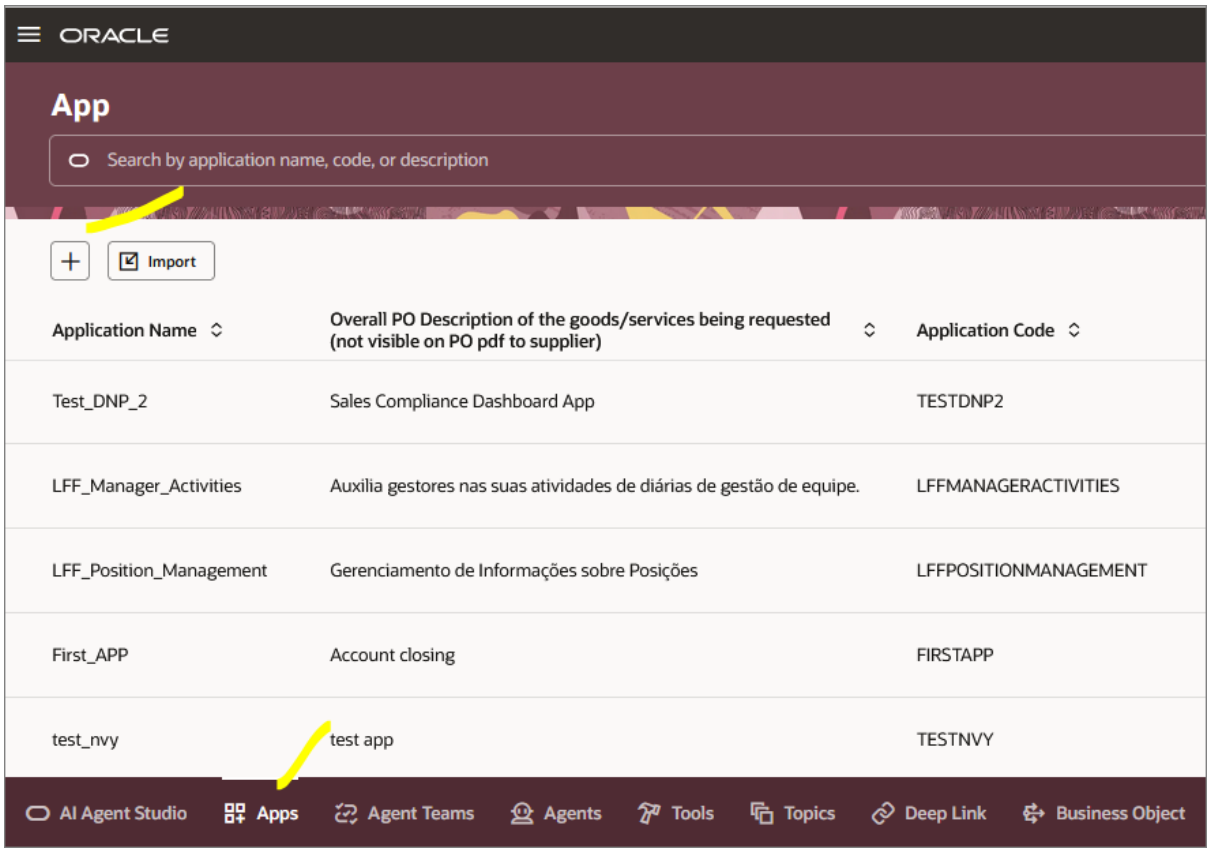
Do not show any other fields in the MultiRecord widget.

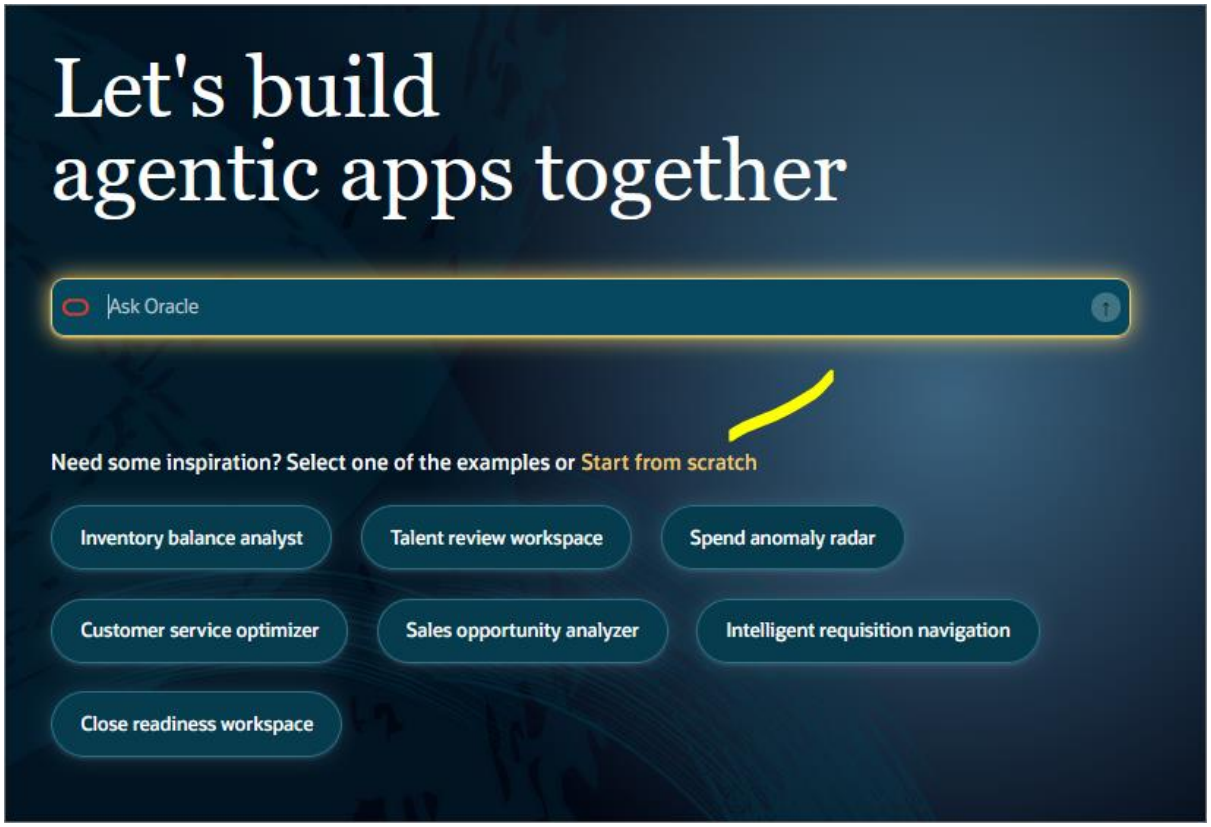
Widget display format:

```
<oraInfoDisplay key="list-1687632000">{
  "title": "Suppliers",
  "patternId": "multiRecordWidget",
  "description": "List of suppliers",
  "properties": {
    "subtitle": "Suppliers",
    "cols": ["Supplier id", "Supplier ", "Supplier" , "SupplierType" , "InactiveDate" , "Status" ],
    "rows": [
  {
    "cells": [
      "<Supplier Id r to be replaced>",
      "<Supplier to be replaced>",
"<SupplierType to be replaced>",
"<InactiveDate to be replaced>",
"<Status to be replaced>",
      "<Supplier Number to be replaced>"
    ]
  }
]
  }
}</oraInfoDisplay>
```

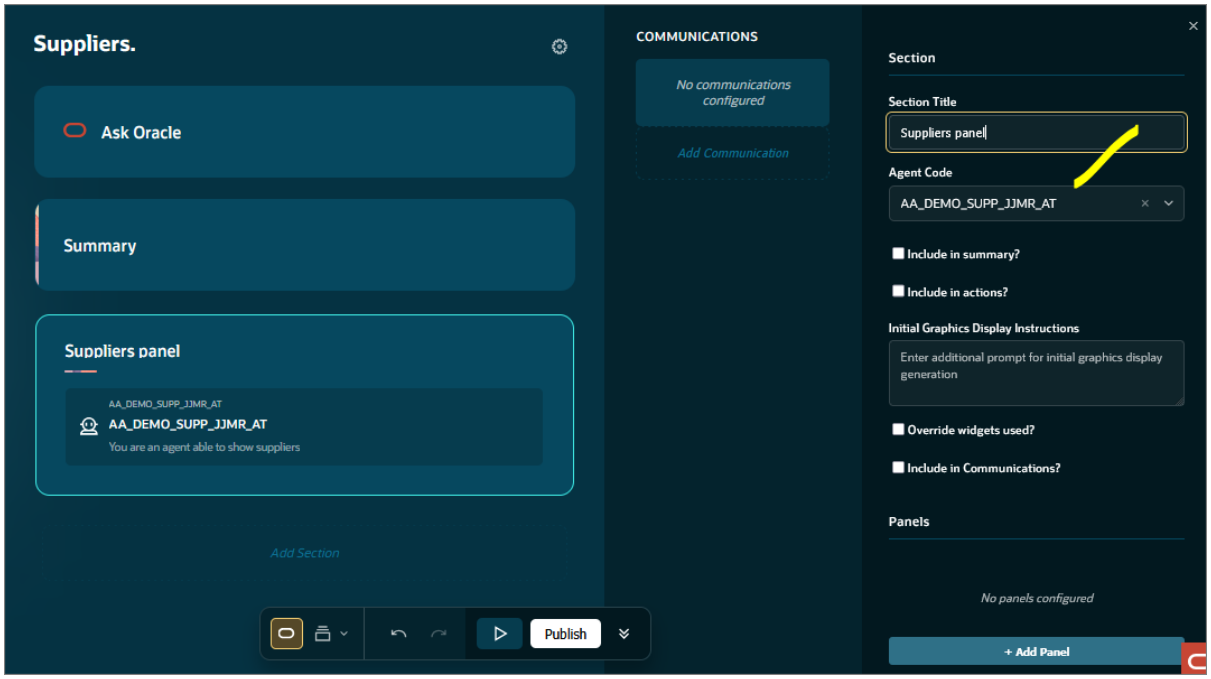
Application.

We have created from scratch as shown.





And include just our Agent Team.

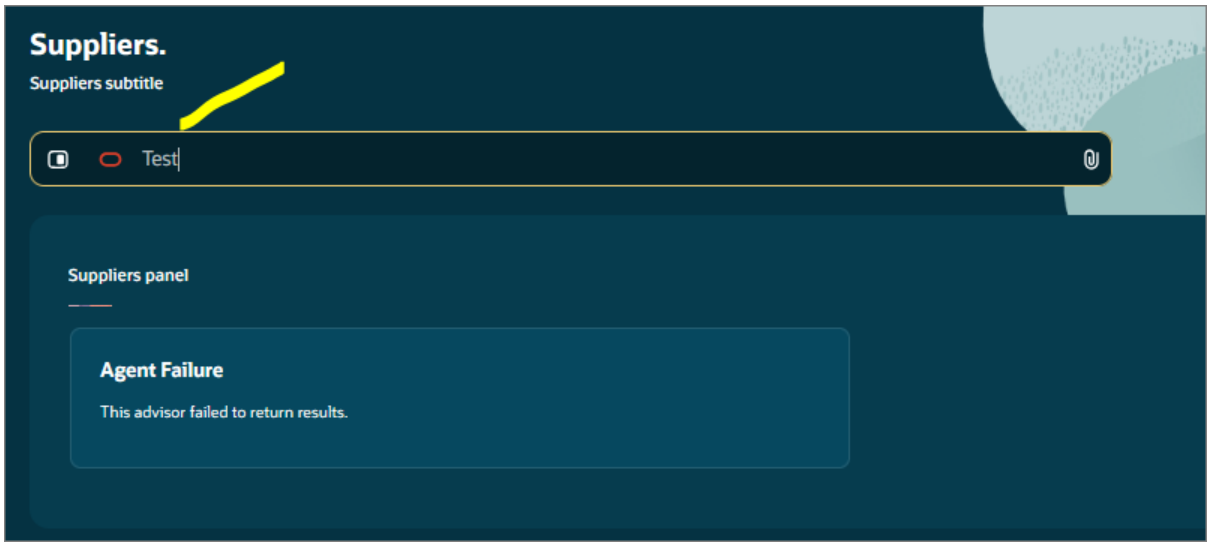


Then we have completed some labels.

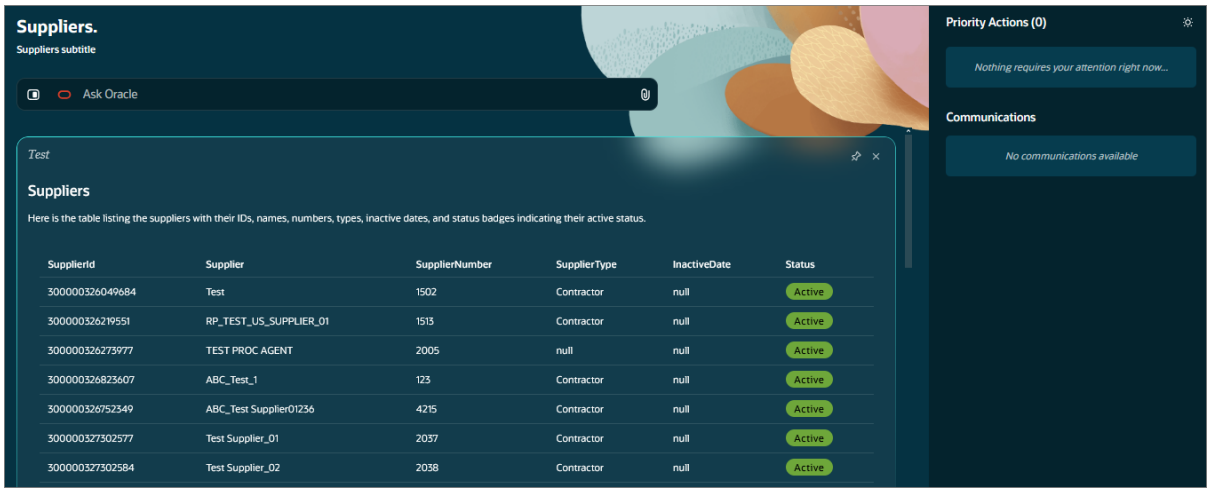
Testing.

Our business object will show up to 25 suppliers using the supplier entered by user in our App.

If we enter “Test” and press “Enter”.



The app. Will return and show a widget filtering suppliers whose name contains “Test” in the name:



It is a basic demonstration of how to wire artifacts with the new 26B features.

Not big deal, but a first approach to get a glimpse of the Widgets features.

More advanced demos in the next chapters.

Conclusion.

We can show data filtered in a custom way in a widget multirecord embedded in an Agent Team included in the Application.

Technical

Code example	Comments
AA_DEMO_SUPP_JJMR_AT.json	Agent team code to import
AA_DEMO_SUPP_JJMR.json	Application code to import